```
      let balanceAfter = Number(web3.utils.fromWei(await
                         web3.eth.getBalance(ACC1), 'ether'));
      assert.isAbove(balanceBefore - balanceAfter, BID1+1);
      assert.isBelow(balanceBefore - balanceAfter, BID1+2);});
```

There are three parts to this bidding function:

1. Set up or initialize before bidding.
2. Enter statements for bidding.
3. Await completion of bidding, and check the results.

Actual code snippets for these operations can be extracted from the blind auction code. Note the initialization performed by the `let` statements. The bidding operation itself is asynchronous because the transactions need time to run and be recorded on the blockchain. So you await the completion of the bidding operation. When the bidding is complete, you can evaluate the results by using assert statements--in this case, `assert.isAbove()` and `assert.isBelow()`.

10.5.2 Executing the blind auction test script

Now you are ready to run the test script and observe its operation. The commands for running the test script are the same as the ones that you used for counter and ballot smart contracts. Download the code blindAuction-contract from the codebase of the chapter. Start the Ganache test chain by clicking its icon and then clicking Quickstart. Then navigate to the blindAuction-contract directory. Use these commands to execute the test script: